

ADB



CyberSecurity

Consultoria

Cliente: LojaZeta

2025-09-29



KENSEI

CYBERSECURITY

1. SUMÁRIO EXECUTIVO

A LojaZeta é uma empresa de e-commerce em expansão que trabalha com um site exposto na internet e um ambiente hospedado em nuvem. Como opera com dados sensíveis de clientes e transações, está naturalmente exposta a ataques digitais.

Recentemente, foram identificadas tentativas de invasão por SQL Injection (injeção de comandos no banco de dados), XSS (injeção de scripts maliciosos no navegador do usuário) e ataques de força bruta na página de login. Hoje, não existe uma ferramenta central para monitorar os eventos de segurança, nem uma forma rápida e organizada de responder a incidentes.

A proposta aqui apresentada tem foco em três pilares principais:

1. Criar uma defesa em camadas que proteja desde o perímetro até dados e identidades.
2. Implantar um monitoramento centralizado e prático, mesmo com baixo orçamento.
3. Estabelecer um plano de resposta a incidentes (padrão NIST) que seja simples de seguir e funcione na vida real.

Tudo é pensado no modelo 80/20: priorizar o que traz o maior impacto com o menor esforço, especialmente nos primeiros 30 dias.

2. ESCOPO E METODOLOGIA

O que está incluído nesta proposta:

- Diagnóstico com base no briefing fornecido.
- Definição de uma arquitetura de segurança em camadas.
- Criação de um plano de monitoramento e SIEM mínimo viável.
- Estruturação de um plano de resposta a incidentes baseado no modelo NIST.
- Proposta de roadmap com ações para 30, 90 e 180 dias.

Como a análise foi conduzida:

A consultoria partiu do cenário descrito do ambiente: aplicação em Node.js servida por Nginx, banco de dados PostgreSQL e infraestrutura em nuvem (IaaS). Considerou também a limitação da equipe (2 devs e 1 ops), a ausência de SIEM e a necessidade de implementar soluções com baixo custo e alta eficiência.

Premissas adotadas:

- O ambiente permite instalação de agentes ou pequenas ferramentas de monitoramento.
- O cliente quer priorizar ações práticas e rápidas (quick wins).
- Há possibilidade de organizar logs e implementar controles básicos sem reestruturar todo o sistema.

3. ARQUITETURA DE DEFESA EM CAMADAS

A segurança será organizada em diferentes camadas. Isso impede que uma única falha cause um incidente grave. Cada camada faz parte de uma “barreira de proteção”.

- **3.1 - Perímetro (Entrada da Aplicação)**

Aqui é onde os acessos externos chegam.

A proposta inclui a instalação de um WAF (Web Application Firewall), como ModSecurity com as regras OWASP CRS. O WAF atua como um filtro que bloqueia tentativas de SQL Injection, XSS e outras explorações antes que elas cheguem na aplicação. Também será aplicado rate limiting para limitar acessos repetitivos (como brute-force).

- **3.2 - Rede**

Nesta camada, o objetivo é controlar o tráfego interno.

Será feita uma segmentação mínima entre a aplicação e o banco de dados, e aplicadas regras de firewall na nuvem para impedir acessos indevidos (por exemplo, SSH ou banco de dados só acessíveis internamente ou por IP autorizado).

- **3.3 - Host (Servidores)**

Envolve proteger o sistema operacional onde a aplicação roda.

As ações incluem atualizações automáticas de segurança, hardening (remoção de serviços e portas desnecessárias) e uso de soluções como auditd ou OSSEC para detectar comportamentos suspeitos.

- **3.4 - Aplicação**

A proteção do código é essencial para evitar que vulnerabilidades sejam exploradas.

Serão adotadas validações de entrada, prepared statements para prevenir SQL Injection e um mecanismo de bloqueio ou atraso após várias tentativas de login (como fail2ban ou rate limit no próprio Nginx).

- **3.5 - Dados**

Aqui entram backups e controle da informação.

O plano inclui manter rotinas de backup com criptografia e testes de restauração (algo que hoje não acontece), além de limitar o acesso ao banco apenas a usuários e serviços autorizados.

- **3.6 - Identidade**

Nesta camada, protege-se o acesso de pessoas e serviços.

Será implementado o uso de senhas fortes, autenticação multifator (MFA) para acessos administrativos, controle de privilégios e remoção de credenciais antigas ou não usadas.

4. MONITORAMENTO & SIEM

Hoje os logs estão espalhados e não existe uma visão central do que acontece. Isso impede a detecção rápida de ataques.

◆ 4.1 Fontes de log essenciais

- Nginx: acessos e erros da aplicação.
- Node.js: registros do comportamento da aplicação.
- PostgreSQL: acessos, consultas e falhas.
- Sistema Operacional: autenticação e processos.

◆ 4.2 Centralização

A proposta é instalar um agente leve (como Filebeat) para enviar os logs para uma ferramenta como ELK (Elasticsearch + Kibana) ou Graylog. Isso permite visualizar tudo em um só painel.

◆ 4.3 Alertas e correlações

Com os logs centralizados, será configurado alerta para:

- Múltiplas falhas de login em pouco tempo.
- Padrões de SQLi ou XSS no tráfego.
- Acessos administrativos fora de horário.
- Erros 500 ou comportamentos anormais no servidor.

◆ 4.4 Indicadores (KPIs)

Com o monitoramento ativo, é possível medir:

- Tempo médio de detecção (MTTD)
- Tempo médio de resposta (MTTR)
- Quantidade de tentativas de ataque bloqueadas
- Percentual de cobertura dos logs essenciais

5. PLANO DE RESPOSTA A INCIDENTES (Modelo NIST)

A resposta a incidentes será organizada em cinco etapas simples e diretas, baseadas no padrão NIST:

✓ 1. Detecção

O SIEM e os alertas identificam atividades anormais. A equipe é notificada.

✓ 2. Contenção

Ação rápida para impedir que o ataque avance.

Exemplo: bloquear IP, isolar servidor, ajustar o WAF temporariamente.

✓ 3. Erradicação

Remover a causa do problema.

Pode envolver troca de senha, remoção de malware ou correção de vulnerabilidade.

✓ 4. Recuperação

Restauração de serviços ao estado normal com validações.

Exemplo: restaurar backup testado ou reativar servidor limpo.

✓ 5. Lições Aprendidas

Documentar o que aconteceu, entender a origem e ajustar controles para que o incidente não se repita.

Também serão criados **runbooks (passo a passo prático)** para os principais riscos identificados:

- SQL Injection
- XSS
- Brute Force em /login
- Indisponibilidade (ataque DoS ou falha de serviço)

6. ROADMAP E PLANO 80/20

O plano é dividido por prioridade e tempo de execução.

◆ Nos primeiros 30 dias (quick wins):

- Ativar o WAF com regras OWASP.
- Começar a centralização de logs (Filebeat + ELK ou Graylog).
- Aplicar rate limiting no login.
- Fazer o primeiro teste de backup.

◆ Em até 90 dias:

- Criar painéis e alertas configurados.
- Implementar MFA nos acessos críticos.
- Melhorar validações na aplicação (XSS/SQLi).
- Escrever os playbooks mais urgentes.

◆ Em até 180 dias:

- Avaliar implantação de IDS/IPS.
- Incluir logs de infraestrutura (nuvem e containers).
- Simular resposta a incidentes para treinar a equipe.

7. RISCOS, CUSTOS E SUPOSIÇÕES

Riscos principais:

- Falta de testes de backup pode atrasar a recuperação.
- Equipe pequena pode limitar a execução rápida.
- Falhas de configuração podem gerar alertas falsos.

Custos estimados:

- Ferramentas open source para minimizar licenças.
- Possível custo com armazenamento e servidor para logs.
- Necessidade de horas técnicas da equipe interna ou consultoria.

Suposições:

- O ambiente permite instalação de ferramentas leves.
- A gestão concorda com foco em prioridades.
- Não serão necessárias grandes mudanças de arquitetura.

8. CONCLUSÃO

A LojaZeta tem um cenário típico de empresa em crescimento: alta exposição na internet e pouca visibilidade sobre o que acontece “por dentro”. As tentativas de ataque já identificadas mostram que os riscos são reais.

Com as medidas propostas — WAF, monitoramento centralizado, respostas padronizadas e segurança em camadas — é possível reduzir drasticamente as chances de invasão e minimizar impactos financeiros e operacionais.

A Consultoria AD3 CyberSecurity recomenda iniciar pelos controles de maior impacto em até 30 dias para construir rapidamente uma base de proteção viável e evoluir gradualmente com baixo custo e alta efetividade.

Próximo passo sugerido: validar o cronograma inicial e dar início às ações do plano 80/20.

